2 - Pointers → Array

T-1 : Start & end → Start++ , end-- →
- Sorted array/string
- Pairs, triplets
- Partitioning

T-2 : Slow & fast pointers →

```
int j=0
for(int i=0 to n)
    if(cond) {j++};
```

→
- find middle
- Cycle detection in linked list or number seq.

Trik →
- Sorted Array
- Have to find pairs (not group like Subarray)

Example → # Leet - 167 Two Sum II → Start & end pointers

- Array is sorted
- Can also solve using Hashmap ↓ T- O(logn)
- T- O(n)
- if (sum > target) end--;
- if (sum < target) start++;

In Leet - 1 Two Sum → Hash map ↓ T- O(logn)

- Array was not sorted So, need to sort first before using 2-pointers → T- O(n logn)

Leet - 977 Square of Sorted Array → Start - End pointer + 1 extra pointer to iterate in answer storing array.

- Array is sorted [-4,-1,0,3,10]
- int $\boxed{S=0}$ $\boxed{e=n-1}$ $\boxed{Pos = n-1}$ → 3 pointers
- if(abs(num[s]) > abs(nums[e])

```
if(abs(num[s]) > abs(nums[e])
        ans[Pos] = num[s] * num[s]
        S++
else
        ans[Pos] = num[e] * num[e]
        e--
Pos--;
```

filling ans vector from back side (left ← right)

abs(-4) < abs(10) ↓ 100
So, store and--;

abs (-4) > abs(3) → 16
So store s++;

(Pos--;) → moves right to left ←

T- O(n)   S- O(n) → storing ans in vector

2nd method →
- store sq in same array by replacing element with their squares
- then sort nums
- T- O(n logn)   S- O(1)

Ajanta PRODUCTS Officer

⌗ Leet-26 Remove Duplicates from Sorted Array → Slow & fast pointer

- * Can't use extra vector to store Answer
- * Have to make updates in original array

```
*   int j=0
    int i=0 to n
       if (num[j] != num[i])
           j++
           num[j] = nums[i];
```

⌗ Leet-15 3-SUM → 3 pointers

start
End

$S - O(2 \times \text{no of pairs})$

* Brute → * 3 nested loops $T - O(n^3)$

* Insert in SET $T - O(\log(\text{no of pairs}))$

Overall → $\boxed{T - O(n^3 \times \log(\text{no of pairs}))}$

- * use set to store unique pairs
- * 3 loops to find pair with sum=0
- * sort elements of pairs before storing into set
- * this way only unique pairs will go into set

```
Set<vector<int>>st                       → set of vectors
for(i=0 to n-2)
  for(k=i+1 to n-1)
    for(j=k+1 to n)
      if(n[i]+n[j]+n[k]==0)
        vector<int> temp = {n[i], n[k], n[j]}   ← Store each found pair in vector
        sort(temp.begin(), temp.end())          ← Sort the elements of pairs stored in vector
        st.insert(temp)                         ← only unique pairs will go into set
return vector<vector<int>> (st.begin(), st.end())
```

* Better → * 2 nested loops $T - O(n^2)$

* Hashset $T - O(\log(\text{no of pair}))$

Overall → $\boxed{T - O(n^2 \times \log(\text{no of pair}))}$

```
 i j
-1, 0, 1, 2, -1, -4
m = -(-1+0)                → [ ] S → empty Hashset
Store j in S & iterate j++

 i     j
-1, 0, 1, 2, -1, -4        → [0] S
m = -(-1+1)=0 → m found in Hashset

So. Store i=-1  m=0  j=1 in vector
then sort the elements of vector
then store unique pair in set 'st'
```

```
Set<vector<int>> st
for (i=0 to n)
  Set<int> S                          → Hash set
  for(j=i+1 to n)
    int m = 0 - (n[i] + n[j])         → target
    if(s.find(m) != S.end())
      vector<int> v = {n[i], m, n[j]}
      sort (v.begin(), v.end())
      st.insert (v);
    S.insert(n[j])                    → Store j into set to quick look of m & then move j ahead
return vector<vector<int>> (st.begin(), st.end());
```

For every new 'i' Hash set will be cleared

* Optimal → * Fix 1 pointer & iterate in loop
* *Start & *End pointer
* T — $O(N\log N) + O(N^2)$

* S — O(no of pairs)

$O(N\log N)$ ↓ Sort

$O(N^2)$ ↓ for each i (loop) we are searching j & k → so nested loop
j ↓ start
k → end

* Sort the array first

```
* for (i=0 to n)
    if(i>0  && num[i] == num[i-1]) Continue;

    int j = i+1;
    int k = n-1;
    while (j<k)
        int s = n[i] + n[j] + n[k]
        if (s>0)  k--;
      else if (s<0)  j++;
        else{
        vector<int> temp = {n[i], n[j], n[k]};
        ans. push_back(temp);
        j++;
        k--;
        while (j<k  && n[j] == n[j-1]) j++;
        while (j<k  && n[k] == n[k+1]) k--;
        }
```

↳ except for 1st element, it will check for rest all elements that if n[i] == n[i-1] same as prev element then skip & move to next i

start, end
i
-4, -4, -1, -1, 0, 1, 2
-4-4+2 < 0
start++

i, s, e
-4, -4, -1, -1, 0, 1, 2
skip
-1-1+2 = 0

found $\boxed{s=0}$ ←

Store pairs in vector ←

Store into ans vector ←

skip duplicates j & k {

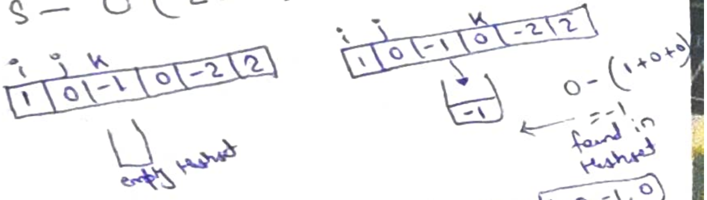
Leet-18 4Sum → (Similar to 3 Sum)

* Brute → * 4 nested loops
T — $O(N^4)$
S — O(2 × no of pairs)

* Better → * 3 nested loops + Hashset
T — $O(n^3 \times \log(m))$
S — $O(2 \times m) + O(N)$

$\boxed{m \to \text{no of Pairs}}$

take long long zero = target − (1LL * n[i] + n[j] + n[k])
1LL → imp



0 − (1+0+0) = −1 found in Hashset

So store $\boxed{1, 0, -1, 0}$ in pair

Sort → $\boxed{-1, 0, 0, 1}$ store in set

* Optimal → * 2 nested loops + 2 end pointers (start, end)
* Sort the array first then fix i & j in nested loops
* for every i & j start++, end--
* long long Sum = 1LL * n[i] + n[j] + n[start] + n[end];
T — $O(N^3)$ S — O(no of Pair)

3 Sum Smaller — Algo Master-259

- Same Concept as optimal Solution of 3 Sum
- T — O(n²)   S — O(1)
- Sort (n.begin() , n.end())

Can't use Hashmap

```
int n = nums.size();
int Count = 0;
for (i = 0 to n-2)
    int left = i+1
    int right = n-1;
    while (left < right)
        int sum = n[i] + n[left] + n[right]
        if (sum < target)
            Count += (right - left)   ——→ all elements from left+1 to right will have sum < target
            left++;
        }
        else{
            right--;
        }
```

Leet- 16   3 Sum Closest

- Same Concept as optimal Soln of 3 Sum
- T — O(n²)   S — O(1)
- int ClosetSum = n[0] + n[1] + n[2]

```
Sort (n.begin() , n.end())
    if (abs (target - sum) < abs (target - closestSum)
        closest Sum = Sum;

    if (s > target) end--
    if (s < target) s++
    if (s = target) return sum;
```

Leet- 75   Sort Colors

Dutch flag (3-pointers)
↓
Low = 0
mid = 0
High = n-1

- 3 pointer ——→ low = 0 (↗ l)   mid = 0   high = n-1 (↳ r)

```
while (mid <= r)
    if (n[mid] == 2)
        Swap (n[mid], n[r])
        r--;
    else if (n[mid] == 0)
        Swap (n[mid], n[l]);
        l++;
        mid++;
    else
        mid++;
```

T — O(n)
S — O(n)

other method ——→ Sorting T—(n log n)

2 pointers – Array

Leet-713 Subarray Product less than K

- * Elements are in range [1, 1000] so no (-)ve number & '0'
- * Leet-152 (max Prod Subarray) → Had (-)ve no. & '0' also So, Had to solve using
  {check miscellinous}

```
maxP = max (num[i], maxP x num[i])
minP = min (num[i], minP x num[i])
ans = max (maxP, ans);
if (num[i] < 0)  ~~swap~~
    swap (maxP, minP);
```

[1, -1, 2, -3] 0 [1, 2, 3, 4] 0 [1, -1, 2, 3]

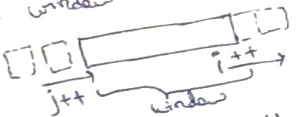
→ jb-jb (-)ve element ayega min wala max & max wala min ban jayega

T - O(n)   S - O(1)

* 2pointer Sliding window pattern

```
int j=0          → Pointer use to remove element from left
int Count=0
int P=1
if(k<=1) return 0        → p should be strictly less than k so there would be no product less than 1 as range is (+)ve

for(i=0 to n)            → ptr to increase the window towards right
    p *= num[i]
while(p>=k && j<=i)
    p = p/num[i]         → jb tak product k se bada hai left side se elements nikalte jao window se using 'j'.
    j++
```



Striver – Med – 9th Leaders in Array

* [10, 22, 12, 3, 0, 6] → 22, 12, 6 are leaders as after them all element are smaller on right side of these elements

6 ↓ element at (n-1)th pos will always be leader

- * use vector or list to store all leaders
- * move from right to left ←

T=O(n)
S-O(n)

```
int j = n-1
L.push-front(arr[j])
for(i=n-2 to 0 ; i--)
    if (arr[i] > arr[j])
        j=i
        L.Push_~~front~~(arr[i])
```

[10, 22, 12, 3, 0, 6] (i at 0, j at 6) → [6]

arr[i] = 12 > 6
So, push-front 12 → [12 | 6]

j = 12
arr[i] = 22 > 12 → [22 | 12 | 6]
j=22
arr[i] = 10 xx not greater than 22

# Striver Med – 10th Longest Consecutive Sequence

→ Better approach : Sorting

→ optimal approach : (Red Black tree) miscellaneous

- * Store all element into a set
- * then for all elements in a set → `for(auto it:S)`
- * Catch the element which is lowest → `if(S.find(it-1)==S.end)`
  ↓ there won't be any preceding no to that element in set
- * After catching lowest no. of sequence (no preceding no should be there → (it-1) should not be in set
- * then find it+1 in iteration in set & increment count : `while(S.find(it+1)!=end)`
  `x = x+1;`
  `Count++;`

Sounds similer to leet-581 but can't be solved using 2 ptrs

Leet-581 Shortest Unsorted Continuous Subarray

Better Soln: → [0 2 4 | 5 3 8 1 9 10 | 6 7]

0 1 2 3 4 5 6 7 8 9

// Find boundaries : —

left to right →

```
{ int low = 0
{ while((low < n-1) && num[low] <= num[low+1]) low++
```

(low < n-1 → iterate low; num[low] <= num[low+1] → until increasing sequence is maintained; low++ → iterate low)

→ So, low will be updated to first index where order is breaking Ex:— low = 2 arr[low] = 5

right to left ←

```
{ int high = n-1
{ while ((high>0) && num[high-1] <= num[high]) high--
```

→ High will be updated to first index from right where order is breaking Ex:— high = 7 num[high] = 10

So, unsorted window → $\boxed{5, 3, 8, 1, 9, 10}$
2 3 4 5 6 7

when low iterated throughout & no sequence break found means whole arry is already sorted

*** Edge → `if(low == n-1) return 0`

// Find max & min elements of unsorted subarray

```
int maxE = INT_MIN;
int minE = INT_MAX;
for(int i=low; i<=high; i++)
    maxE = max(maxE, num[i])
    minE = min(minE, num[i])
```

// Now extend boundaries on left & right to check if more no to be included.

2, 4, $\boxed{5\ 3\ 8\ 1\ 9\ 10}$ 6, 7 (3 → min, 10 → max)

`while(low-1 >= 0 && num[low-1] > minE) low--` → 4 is greater than 3 so, will be included in unsorted array window

`while(high+1 < n && num[high+1] < maxE) high++` → 6, 7 are smaller than 10 so will be included

// Find Ans → $\boxed{high - low + 1}$

$\boxed{T - O(n)}$ $\boxed{S - O(1)}$

Optimal Sol? T = O(n) S = O(1)

→ In prev approach we finding boundaries then finding min-max and then updating boundaries comparing current elements with min & max

→ Here we follow direct intuition :—

2 Rules:—

i) → Iterate from left to Right (→)
→ find max Element
& number Going to the right side of maxE will become new upper boundary if it less than maxE

```
int high=0
for (int i=0 to n)
maxE = max(maxE, nums[i])
if (num[i] < maxE)
            high = i
```

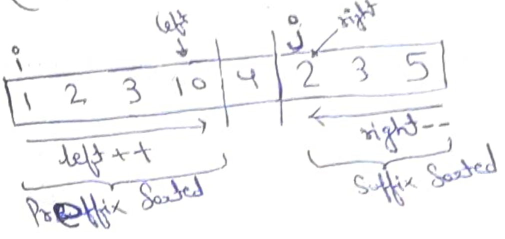
★★
```
// Already Sorted
if (high == 0) return 0;
```

ii) Iterate from right to left (←)
find minElement
& number Going on left side of minE will become new lower boundary, if it is greater than minE

```
int low = n-1
for(int i=n-1 to 0; i--)
minE = min(minE, num[i])
if (num[i] > minE)
    low = i
```

Leet-1574 Shortest Subarray to be Removed to Make array Sorted

Better : T - O(n) S - O(1)



```
// Finding Sorted prefix
int left=0
while (left < n-1 && arr(left) <= arr[left+1]) left++;
```

→ left will stop where ascending order breaks so arr[left] = 10

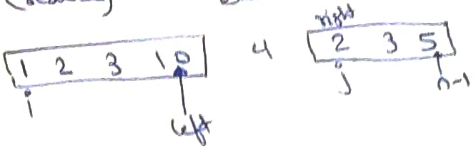
Edge → if whole array is already sorted, left will iterate fully upto n-1 so, if (left == n-1) return 0;

```
// Find Sorted Suffix
int right = n-1
while(right > 0 && arr[right-1] <= arr[right]) right--;
```

// Case 1: either
→ Keep only sorted prefix → Delete everything after left → Part deleted = (n-left-1)
→ Keep only sorted suffix → Delete all upto right → part deleted = (right

So, int ans = min(n-left-1, right);
↳ min deleted part

// Case 2: Improve (reduce) deleted portion by merging prefix & suffix & delete elements b/wn i & j



iter 1 : Sorted → Pre $\boxed{1}$ Suffix $\boxed{2\ 3\ 5}$
delete → $\boxed{2\ 3\ 10}$ 4 = 4 elements

iter 2 : Sorted → Pre $\boxed{1\ 2}$ Suf $\boxed{2\ 3\ 5}$
delete → $\boxed{3\ 10}$ 4 = 3 elements

iter 3 : Sorted → Pre $\boxed{1\ 2\ 3}$ Suf $\boxed{3\ 5}$
delete → $\boxed{10}$ 4 $\boxed{2}$ = 3 elements

iter 4 : Sorted → $\boxed{1\ 2\ 3\ 10}$
delete → 4 $\boxed{2\ 3\ 5}$ = 4 elements
now i can't be move further as order will break 10>4

```
int i=0
int j= right
while(i <= left  && j<n)
    if (arr[i] <= arr[j])
        ans = min(ans, j-i-1);
        i++
    else  j++
```

j-i-1 → remove elements b/wn i & j

Optimal : T- O(n)   S- O(1)   slightly improving prev code

```
// iterate left to right to find initial pt. of suffix sorted part
    int j= n-1;
    while (j>0  &&  arr[j] >= arr[j-1]) j--

// Edge Case
    int ans = j;
```

→ remove all elements upto j & keep only suffix part so if j iterated full upto 0 then no need to delete anything as full sorted so ans = j = 0 in edge case

```
// Enhance by including elements from prefix sorted & merge with suffix sorted
    int    i=0
    while (i<j   &&  (i==0 || arr[i] >= arr[i-1])
    {
        while(j<n  &&  arr[j] < arr[i])→{j++}

        ans = min(ans, j-i-1)
        i++
    }
```

arr[i] >= arr[i-1] → this will stop i to going to 4 as 10>4 (i-1, i)

↓ will throw overflow at i=0
↳ so included this part

⌗ Leet - 202   HAPPY NUMBER

(miscellanious) → Brute
T- O(log n)
S- O(log n)

(Floyd's Cycle) → Tortoise & Hare → optimal
T- O(log n)
S- O(1)

→ Check this sol^n in Floyd page

n=19 → $1^2+9^2=82$
$8^2+2^2=68$
$6^2+8^2=100$
$1^2+0^2+0^2=1$ ✓✓

n=2 → 2→4→16→37→58→89→145→42→20→4→16 ---- Cycle repeats

n=7 → 7→49→97→130→10→1 ✓✓

```
unorderedset<int> seen
while (n !=1    &&   !seen.count(n))
    seen.insert(n);
    while(n>0)
    { int digit = n%10
         sum += digit * digit
         n /=10
    }
    n=sum
return  n==1;
```

→ terminate if get n=1 or if same digits start repeating (loop)

→ insert every new upcoming n into set

} 19 = $1^2+9^2=82$ Calculate sum

19 → 1^2+9^2 → 82 new n

n=sum → update n with new value

return n==1; → HAPPY HAPPY HAPPY

③

2-pointers Array

Leet-11 Container with Most Water

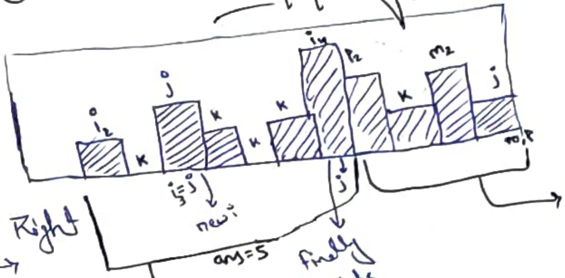
* 2 end pointers → s = 0
e = n-1

* a = min(height[s], height[e]) * (e-s);

Condition →
```
if (height[s] < height[e])
    s++;
else e--;
```

$\boxed{T - O(n)}$ $\boxed{S - O(1)}$

Leet-42 Trapping Rain Water (Stack / optimal)



new i

ans=5

1st pass, water of this part calculated

finally i stops Here as j iterated fully

2nd pass water of this part will be Calculated
ans=1

total ans=6.

2 pointer (optimal) →

1st pass left to Right →

```
i = 0
j = 0
ans = 0
while (j <= n-1)
    if (h[j] >= h[i])
        for (int k = i to j)
            ans = ans + (h[i] - h[k]);
        i = j;
    j++;
```

j will iterate fully →

find a block equal to or greater than 1st block to hold water ←

Calculating water holded by 2 blocks

$\boxed{T - O(n) \quad S - O(1)}$

2nd pass Right to Left ← (Same as left to Right)

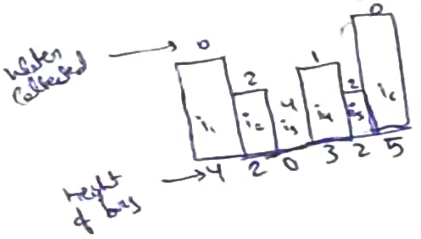
```
m = n-1
p = n-1
[s = i]
while (p >= s)
    if (h[p] >= h[m])
        for (int k = m to p)
            ans = ans + (h[m] - h[k])
        m = p
    p--;
```

will be iterated upto where i stopped ←

Ajanta PRODUCTS Officer

* Check brute force Code on leetcode (2 pointer) → O(n^2)

Basic Algo → Water Stored over a particular Block = * $\boxed{\min(\text{leftMax}, \text{Right max}) - h[i]}$



for i_4 bar → left max = 4 (i_1)
Right max = 5 (i_6)

min (4,5) = 4
water = 4 − h[i_4]
= 4 − 3 = ①

Now we can calculate left Max & Right max in 3 different ways

* Brute → using Built in fn Calculate for each element again & again forming nested loops → T = O(n^2), S − O(1)

* Better → Store left Max & Right max for every element in 2 diff vectors → T = O(n), S − O(2n)
(left max, Right max)

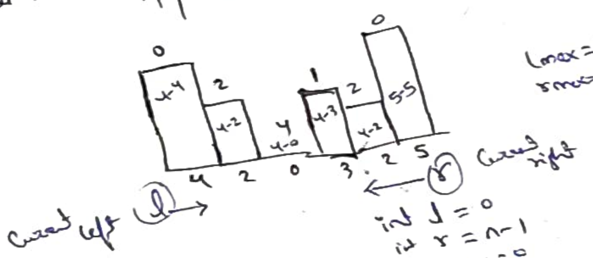
Prefix max / Suffix max Approach →

```
vect<int> lmax (n,0)
vect<int> rmax (n,0)
lmax[0] = ht[0]
rmax[n-1] = ht[n-1]
for (i=1 to n)
    lmax[i] = max (lmax[i-1], ht[i])
for (i=n-2 to 0)
    rmax[i] = max (rmax[i+1], ht[i])
```

then apply min (left max, Right max) − h[i]) in a loop

Best Optimal 2 pointer Approach

T = O(n)   S − O(1)



lmax = 4 // deciding factor as lmax < rmax
rmax = 5

water = lmax − Current block height

```
int l = 0
int r = n-1
int lmax = 0
int rmax = 0

while (l < r)
    lmax = max (lmax, height[l])
    rmax = max (rmax, heigh[r])
    if (lmax < rmax)
        ans += lmax - height[l];
        l++;
    else
        ans += rmax - heigh[r];
        r--;
```

current from left
current from right
for current left lmax will decide as it's smaller
water over current right will be decided by rmax as it's smaller